

Modül 14

State Management

NgRx, Signal Store, Patterns

Modül: 14 / 30

Ders Sayısı: 7 ders

Seviye: İleri

Versiyon: Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 14'e Hoş Geldin

Büyük uygulamalarda state'i nasıl yönetiriz? Service-based state, NgRx (Redux pattern), @ngrx/signals (modern), ve hangisini ne zaman seçmeliyiz?

Öğrenecekler:

- ✓ State management neden gerekli?
- ✓ Service-based state (basit uygulamalar için)
- ✓ NgRx (Store, Actions, Reducers, Effects, Selectors)
- ✓ NgRx Component Store
- ✓ @ngrx/signals (modern Signal Store)
- ✓ State persist (localStorage, IndexedDB)
- ✓ Hangi yaklaşım ne zaman?

State Management Neden Gerekli?

Önce problemi anlayalım.

❏ Problem

Küçük uygulamalarda state component'lerde tutulur. Ama uygulama büyüyünce:

- ✗ Prop drilling: $A \rightarrow B \rightarrow C \rightarrow D$ component'ler arası input geçişi
- ✗ Data duplication: Aynı veri birden fazla yerde
- ✗ Sync sorunu: Bir yerde değişen veri başka yerde eski kalır
- ✗ Side effect karmaşası: HTTP, localStorage, websocket nerede yönetilecek?
- ✗ Test zorluğu: Component'ler birbirine bağımlı

❏ Çözüm — Centralized State

State'i tek yerde tut (store), her component oradan oku ve değiştir.

❏ Single Source of Truth

Verinin tek bir kaynağı olur. Hangi component'in ihtiyacı olursa oradan alır. Değişiklik tek noktada yapılır, tüm bağlı yerler otomatik update.

❏ Yaklaşımlar

Yaklaşım	Karmaşıklık	Use Case
Service + signal()	Düşük	< 10 component, basit state
Component Store	Orta	Feature-specific state
@ngrx/signals	Orta	Modern, signal-first
NgRx Store	Yüksek	Büyük enterprise app

Service-Based State (Önce Bunu Dene)

Çoğu uygulama için yeterli. NgRx'e geçmeden önce buradan başla.

□ Pattern

```
@Injectable({ providedIn: 'root' })
export class TodoStore {
  // === STATE ===
  private _todos = signal<Todo[]>([]);
  private _filter = signal<Filter>('all');
  private _loading = signal(false);
  private _error = signal<string | null>(null);

  // === SELECTORS (Public Read) ===
  readonly todos = this._todos.asReadonly();
  readonly filter = this._filter.asReadonly();
  readonly loading = this._loading.asReadonly();
  readonly error = this._error.asReadonly();

  // Derived state
  readonly filteredTodos = computed(() => {
    const all = this._todos();
    const f = this._filter();
    if (f === 'active') return all.filter(t => !t.completed);
    if (f === 'completed') return all.filter(t => t.completed);
    return all;
  });

  readonly stats = computed(() => ({
    total: this._todos().length,
    active: this._todos().filter(t => !t.completed).length,
    completed: this._todos().filter(t => t.completed).length,
  }));
}
```

```

// === ACTIONS ===
private http = inject(HttpClient);

async load() {
  this._loading.set(true);
  this._error.set(null);
  try {
    const todos = await firstValueFrom(
      this.http.get<Todo[]>('/api/todos')
    );
    this._todos.set(todos);
  } catch (e: any) {
    this._error.set(e.message);
  } finally {
    this._loading.set(false);
  }
}

add(todo: CreateTodo) {
  this._todos.update(arr => [...arr, {
    ...todo,
    id: crypto.randomUUID()
  }]);
}

remove(id: string) {
  this._todos.update(arr => arr.filter(t => t.id !== id));
}

toggle(id: string) {
  this._todos.update(arr =>
    arr.map(t =>
      t.id === id ? { ...t, completed: !t.completed } : t
    )
  );
}

setFilter(filter: Filter) {
  this._filter.set(filter);
}
}

```

⚡ Effect ile localStorage Sync

```
@Injectable({ providedIn: 'root' })
export class TodoStore {
  private _todos = signal<Todo[]>([]);

  constructor() {
    // Load from localStorage on init
    const saved = localStorage.getItem('todos');
    if (saved) {
      try {
        this._todos.set(JSON.parse(saved));
      } catch {}
    }

    // Auto-save on every change
    effect(() => {
      localStorage.setItem('todos', JSON.stringify(this._todos()));
    });
  }
}
```

@ngrx/signals — Modern Signal Store

NgRx ekibinin signal-first store kütüphanesi. Production-ready.

□ Setup

```
// Installation
// npm install @ngrx/signals

import {
  signalStore, withState, withComputed,
  withMethods, withHooks, patchState
} from '@ngrx/signals';

interface TodoState {
  todos: Todo[];
  filter: Filter;
  loading: boolean;
}

const initialState: TodoState = {
  todos: [],
  filter: 'all',
  loading: false,
};

export const TodoStore = signalStore(
  { providedIn: 'root' },

  // State
  withState(initialState),

  // Computed signals
  withComputed(({ todos, filter }) => ({
    filteredTodos: computed(() => {
      const f = filter();
      const all = todos();
      if (f === 'active') return all.filter(t => !t.completed);
      if (f === 'completed') return all.filter(t => t.completed);
      return all;
    }),
    stats: computed(() => ({
      total: todos().length,
      active: todos().filter(t => !t.completed).length,
      completed: todos().filter(t => t.completed).length,
    })),
  })),
);
```

```

// Methods (actions)
withMethods((store, http = inject(HttpClient)) => ({
  async load() {
    patchState(store, { loading: true });
    const todos = await firstValueFrom(
      http.get<Todo[]>('/api/todos')
    );
    patchState(store, { todos, loading: false });
  },

  add(todo: CreateTodo) {
    patchState(store, state => ({
      todos: [...state.todos, {
        ...todo, id: crypto.randomUUID()
      }]
    }));
  },

  remove(id: string) {
    patchState(store, state => ({
      todos: state.todos.filter(t => t.id !== id)
    }));
  },

  toggle(id: string) {
    patchState(store, state => ({
      todos: state.todos.map(t =>
        t.id === id ? { ...t, completed: !t.completed } : t
      )
    }));
  },

  setFilter(filter: Filter) {
    patchState(store, { filter });
  },
})),

// Hooks
withHooks({
  onInit(store) {
    store.load();
  },
}),
);

```

□ Component'te Kullanım

```
@Component({...})
export class TodoListComponent {
  store = inject(TodoStore);

  // State - signal'lar otomatik
  todos = this.store.todos;
  loading = this.store.loading;

  // Computed
  filteredTodos = this.store.filteredTodos;
  stats = this.store.stats;

  // Actions
  add(todo: CreateTodo) { this.store.add(todo); }
  remove(id: string) { this.store.remove(id); }
  toggle(id: string) { this.store.toggle(id); }
}
```

⚡ @ngrx/signals Avantajları

- ✓ Signal-first — modern Angular ile uyumlu
- ✓ Type-safe — TypeScript inference süper
- ✓ Boilerplate az — NgRx Store'a göre %70 daha az kod
- ✓ Tree-shakable
- ✓ DevTools desteği
- ✓ Custom features (withEntities, withDevtools, vs.)

withEntities — Entity Management

List + CRUD pattern'i için NgRx Signal Store entity helper.

□ Entity Pattern

```
import { withEntities, setAllEntities, addEntity, updateEntity, removeEntity } from
 '@ngrx/signals/entities';

export const UserStore = signalStore(
  { providedIn: 'root' },

  withEntities<User>(), // user entity'leri için

  withComputed(({ entities }) => ({
    activeUsers: computed(() => entities().filter(u => u.active)),
    totalUsers: computed(() => entities().length),
  })),

  withMethods((store, http = inject(HttpClient)) => ({
    async loadAll() {
      const users = await firstValueFrom(http.get<User[]>('/api/users'));
      patchState(store, setAllEntities(users));
    },

    addUser(user: User) {
      patchState(store, addEntity(user));
    },

    updateUser(id: string, changes: Partial<User>) {
      patchState(store, updateEntity({ id, changes }));
    },

    removeUser(id: string) {
      patchState(store, removeEntity(id));
    },
  })),
);
```

Classical NgRx Store — Redux Pattern

Geleneksel NgRx (action/reducer pattern). Büyük enterprise app'lerde hala kullanılır.

□ Building Blocks

```
// 1. ACTIONS
export const TodoActions = createActionGroup({
  source: 'Todo',
  events: {
    'Load': emptyProps(),
    'Load Success': props<{ todos: Todo[] }>(),
    'Load Failure': props<{ error: string }>(),
    'Add': props<{ todo: CreateTodo }>(),
    'Toggle': props<{ id: string }>(),
    'Remove': props<{ id: string }>(),
  },
});
```

```
// 2. STATE & REDUCER
interface TodoState {
  todos: Todo[];
  loading: boolean;
  error: string | null;
}

const initialState: TodoState = {
  todos: [],
  loading: false,
  error: null,
};

export const todoReducer = createReducer(
  initialState,
  on(TodoActions.load, state => ({
    ...state, loading: true, error: null
  })),
  on(TodoActions.loadSuccess, (state, { todos }) => ({
    ...state, todos, loading: false
  })),
  on(TodoActions.loadFailure, (state, { error }) => ({
    ...state, loading: false, error
  })),
  on(TodoActions.add, (state, { todo }) => ({
    ...state,
    todos: [...state.todos, { ...todo, id: crypto.randomUUID() }]
  })),
  on(TodoActions.toggle, (state, { id }) => ({
    ...state,
    todos: state.todos.map(t =>
      t.id === id ? { ...t, completed: !t.completed } : t
    )
  })),
);
```

```

// 3. SELECTORS
export const selectTodoState =
  createFeatureSelector<TodoState>('todo');

export const selectTodos = createSelector(
  selectTodoState, s => s.todos
);

export const selectActiveTodos = createSelector(
  selectTodos, todos => todos.filter(t => !t.completed)
);

export const selectLoading = createSelector(
  selectTodoState, s => s.loading
);

// 4. EFFECTS (async)
@Injectable()
export class TodoEffects {
  private actions$ = inject(Actions);
  private http = inject(HttpClient);

  load$ = createEffect(() =>
    this.actions$.pipe(
      ofType(TodoActions.load),
      switchMap(() =>
        this.http.get<Todo[]>('/api/todos').pipe(
          map(todos => TodoActions.loadSuccess({ todos })),
          catchError(error =>
            of(TodoActions.loadFailure({ error: error.message }))
          )
        )
      )
    )
  );
}

```

□ Component'te

```
@Component({...})
export class TodoListComponent {
  private store = inject(Store);

  // Selectors → signal'a
  todos = this.store.selectSignal(selectTodos);
  activeTodos = this.store.selectSignal(selectActiveTodos);
  loading = this.store.selectSignal(selectLoading);

  ngOnInit() {
    this.store.dispatch(TodoActions.load());
  }

  add(todo: CreateTodo) {
    this.store.dispatch(TodoActions.add({ todo }));
  }

  toggle(id: string) {
    this.store.dispatch(TodoActions.toggle({ id }));
  }
}
```

⚠ NgRx Store Ne Zaman?

📌 Karar

Modern Angular'da @ngrx/signals daha popüler. Klasik NgRx Store hala kullanılır ama boilerplate çok fazla. Yeni proje başlatıyorsan signal store tercih et.

State Persistence

localStorage, sessionStorage, IndexedDB ile state'i kalıcı yap.

□ localStorage Persistence

```
import { withHooks } from '@ngrx/signals';
import { effect } from '@angular/core';

const STORAGE_KEY = 'todoState';

export const TodoStore = signalStore(
  withState(initialState),
  withMethods(/* ... */),

  withHooks({
    onInit(store) {
      // Load from storage
      const saved = localStorage.getItem(STORAGE_KEY);
      if (saved) {
        try {
          patchState(store, JSON.parse(saved));
        } catch {}
      }

      // Auto-save
      effect(() => {
        const state = getState(store);
        localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
      });
    },
  }),
);
```

□ Partial Persistence

```
// Sadece bazı field'ları persist et
effect(() => {
  const state = getState(store);
  const toSave = {
    todos: state.todos,
    filter: state.filter,
    // loading, error persist edilmez
  };
  localStorage.setItem(STORAGE_KEY, JSON.stringify(toSave));
});
```

Hangi Yaklaşım Ne Zaman?

Karar verme rehberi.

❏ Karar Ağacı

❏ Hangisini Seçeyim?

State sadece bir component'te mi?

→ Component signal'ları yeterli

2-3 component arasında paylaşım mı?

→ Service-based state (signal + asReadonly)

Birden fazla feature, kompleks state mi?

→ @ngrx/signals (modern, önerilen)

Enterprise, redux pattern'e zaten alışkın mısın?

→ NgRx Store (klasik)

Feature-specific kompleks state mi?

→ NgRx Component Store

❏ Comparison

Konu	Service + Signal	@ngrx/signals
Boilerplate	Minimal	Az
DevTools	Manuel	✓ Var
Type safety	✓	✓✓
Learning curve	Kolay	Orta
Best for	Küçük-orta	Orta-büyük

❏ Genel Tavsiyem

- ✓ Başla: Service + signal()
- ✓ İhtiyaç doğunca: @ngrx/signals
- ✓ Sadece gerekirse: Klasik NgRx Store
- ✓ Premature optimization yapma: İhtiyacın olmadan kütüphane ekleme

Modül 14 Özeti

Neler Öğrendin?

- ✓ State management neden gerekli
- ✓ Service-based state (basit yaklaşım)
- ✓ @ngrx/signals (modern store)
- ✓ withEntities pattern
- ✓ Klasik NgRx Store (Actions, Reducers, Effects, Selectors)
- ✓ State persistence (localStorage)
- ✓ Hangi yaklaşım ne zaman

Sıradaki Modül

Modül 15 — Animations. Angular Animations API, transition'lar, route animations, ve modern alternatifler (CSS, View Transitions API).